Ejercicios de Programación

en JavaScript

Guía práctica para principiantes

Temas cubiertos:

Variables · Tipos de datos · Objetos · Destructuring
Template Strings · Arrays · Operadores lógicos
Condicionales · Métodos de Arrays · Iteradores

Progresión de dificultad

Muy Fácil → **Fácil** → **Medio** → **Intermedio**

Cátedra de Programación • 2026

Índice

Antes de comenzar 2

1. Nivel Muy Fácil 3

2. Nivel Fácil 6

3. Nivel Medio 9

4. Nivel Intermedio 13

5. Juego final • ¿Qué imprime este código? 18

Referencia rápida 21

Antes de comenzar

Esta guía contiene ejercicios progresivos para practicar los fundamentos de **JavaScript**. Cada ejercicio tiene una pista disponible si te trabás y una solución de referencia al final.

> 💡 **Consejo:** intentá resolver cada ejercicio antes de leer la pista o la solución. El error es parte del aprendizaje. Si el código no funciona a la primera, *eso es completamente normal*.

¿Dónde ejecutar el código?

Podés probar todos los ejercicios en cualquiera de estos lugares:

- La **consola del navegador** (F12 → pestaña Console).
- Editores online como **CodePen**, **JSFiddle** o **PlayCode**.

Nivel	Qué practicás
Muy Fácil	Variables, tipos de datos y template strings.
Fácil	Objetos, destructuring y condicionales básicas.
Medio	Arrays, métodos de arrays e iteradores.
Intermedio	Lógica compleja combinando varios temas.
Juego	Desafío integrador: ¿qué imprime este código?

Nivel Muy Fácil

Los ejercicios de este nivel cubren los fundamentos absolutos: declarar variables, distinguir tipos de datos y armar mensajes con template strings.

Ejercicio 1 • Mi primera variable

Muy Fácil | let / const | console.log

Enunciado

Declará una variable `nombre` con tu nombre y una constante `edad` con tu edad. Mostrá ambos valores en la consola.

Salida esperada

```
Lucas
25
```

Código inicial

```
// Declará nombre con let y edad con const
// Luego mostralos en consola
```

`let` vs `const`
Usá `let` cuando el valor pueda **cambiar** más adelante, y `const` cuando el valor sea **fijo**. Evitá `var` (es la forma vieja de declarar variables).

Pista: La sintaxis es `let nombreVariable = valor;` . Para mostrar en consola: `console.log(nombreVariable);` .

Solución

```
let nombre = "Lucas";
const edad = 25;

console.log(nombre);
console.log(edad);
```

Ejercicio 2 • ¿De qué tipo es?

Muy Fácil | typeof | Tipos primitivos

Enunciado

Declará cuatro variables con valores de tipos diferentes (string, number, boolean y undefined) y mostrá en consola el tipo de cada una usando `typeof`.

Salida esperada

```
string
number
boolean
undefined
```

Código inicial

```
let texto     = "Hola";
let numero    = 42;
let booleano  = true;
let nada;

// Mostrá el typeof de cada una
```

Pista: `typeof` es un operador, no una función. Se usa así: `console.log(typeof texto);`.

Solución

```
let texto     = "Hola";
let numero    = 42;
let booleano  = true;
let nada;

console.log(typeof texto);     // string
console.log(typeof numero);    // number
console.log(typeof booleano);  // boolean
console.log(typeof nada);      // undefined
```

Ejercicio 3 • Saludo con template string

Muy Fácil | Template strings | Concatenación

Enunciado

Dadas las variables `nombre` y `edad`, mostrá un mensaje usando **template strings** (comillas invertidas):

```
Hola, me llamo Ana y tengo 30 años.
```

Código inicial

```
const nombre = "Ana";
const edad   = 30;

// Construí el mensaje usando ‘template strings‘
```

Template strings

Se escriben con comillas invertidas (‘) y permiten insertar variables con `${variable}`. Es mucho más legible que concatenar con `+`.

`‘Hola ${nombre}‘` vs. `"Hola " + nombre`

Pista: La tecla de comillas invertidas está generalmente debajo de `Esc` o al lado del `1`.

Solución

```
const nombre = "Ana";
const edad   = 30;

console.log(‘Hola, me llamo ${nombre} y tengo ${edad}
    años.‘);
```

Nivel Fácil

Ahora trabajamos con **objetos** y **destructuring**, dos pilares de JavaScript moderno, junto con condicionales básicas.

Ejercicio 4 • Ficha de usuario

Fácil | Objetos | Template strings

Enunciado

Creá un objeto `usuario` con las propiedades `nombre`, `apellido` y `ciudad`. Después mostrá en consola un mensaje de presentación usando esos datos.

Salida esperada

```
Soy María Pérez y vivo en Córdoba.
```

Código inicial

```
// Creá el objeto usuario y mostrá el mensaje
const usuario = {
  // completá las propiedades
};
```

Pista: Para acceder a una propiedad usá el punto: `usuario.nombre`. Combinálo con un template string.

Solución

```
const usuario = {
  nombre: "María",
  apellido: "Pérez",
  ciudad: "Córdoba"
};

console.log(`Soy ${usuario.nombre} ${usuario.apellido} y vivo en ${usuario.ciudad}.`);
```

Ejercicio 5 • Destructuring de un objeto

Fácil | Destructuring | Objetos

Enunciado

Dado el objeto `producto`, extraé sus propiedades en variables individuales usando **destructuring** y mostrá un mensaje con el formato:

```
Notebook cuesta $850000 (categoría: Tecnología)
```

Código inicial

```
const producto = {
  nombre: "Notebook",
  precio: 850000,
  categoria: "Tecnología"
};

// Usá destructuring para extraer las propiedades
```

Destructuring de objetos

Permite extraer varias propiedades de un objeto en una sola línea:

`const { nombre, precio } = producto;`

Es equivalente a hacer `const nombre = producto.nombre;` y `const precio = producto.precio;` pero mucho más corto.

Pista: Los nombres entre llaves **deben coincidir** con las propiedades del objeto.

Solución

```
const producto = {
  nombre: "Notebook",
  precio: 850000,
  categoria: "Tecnología"
};

const { nombre, precio, categoria } = producto;

console.log(`${nombre} cuesta $${precio} (categoría:
    ${categoria})`);
```

Ejercicio 6 • ¿Mayor de edad?

Fácil | if / else | Operadores

Enunciado

Dada una variable `edad`, mostrá en consola si la persona es mayor o menor de edad.

Ejemplos de salida esperada

Si `edad = 20`: `Sos mayor de edad.`
Si `edad = 15`: `Sos menor de edad.`

Código inicial

```
let edad = 20;

// Completá con un if / else
```

Pista: La estructura es `if (condicion) { ... } else { ... }`. El operador para "mayor o igual que" es `>=`.

Solución

```
let edad = 20;

if (edad >= 18) {
  console.log("Sos mayor de edad.");
} else {
  console.log("Sos menor de edad.");
}
```

Nivel Medio

Empezamos a usar **arrays**, **métodos de arrays** e **iteradores**. Estos temas son la base de prácticamente todo JavaScript moderno.

Ejercicio 7 • Recorrer un array

Medio | for of | Arrays

Enunciado

Dado un array de frutas, mostrá cada una en consola usando un bucle `for...of`.

Salida esperada

```
manzana
banana
naranja
pera
```

Código inicial

```
const frutas = ["manzana", "banana", "naranja", "pera"];

// Recorré el array con for...of
```

Tres formas de recorrer un array

- `for (let i = 0; i < arr.length; i++)` — el clásico, te da el índice.
- `for (const item of arr)` — más simple, te da directamente el valor.
- `arr.forEach(item => ...)` — estilo funcional con callback.

Pista: La sintaxis es `for (const fruta of frutas) { ... }`.

✔ **Solución**

```
const frutas = ["manzana", "banana", "naranja", "pera"];

for (const fruta of frutas) {
  console.log(fruta);
}
```

✕ Ejercicio 8 • Duplicar números con map

Medio map Arrow function

Enunciado

Dado un array de números, creá un **nuevo array** donde cada número esté multiplicado por 2, usando el método `map`.

Salida esperada

```
[2, 4, 6, 8, 10]
```

Código inicial

```
const numeros = [1, 2, 3, 4, 5];

// Usá .map() para crear un nuevo array con cada número x 2
```

`map` vs `forEach`

Ambos recorren un array, pero `map` **devuelve un nuevo array** con el resultado de cada iteración. `forEach` no devuelve nada, solo ejecuta una acción por cada elemento.

Pista: `const dobles = numeros.map(n => n * 2);`

✔ Solución

```
const numeros = [1, 2, 3, 4, 5];

const dobles = numeros.map(n => n * 2);

console.log(dobles); // [2, 4, 6, 8, 10]
```

Ejercicio 9 • Filtrar mayores de edad

Medio | filter | Objetos en arrays

Enunciado

Dado un array de personas (objetos con `nombre` y `edad`), creá un nuevo array que contenga **solo** las personas mayores de edad.

Salida esperada

```
[
{ nombre: 'Ana', edad: 22 },
{ nombre: 'Pedro', edad: 35 }
]
```

Código inicial

```
const personas = [
  { nombre: "Ana",   edad: 22 },
  { nombre: "Luis",  edad: 15 },
  { nombre: "Pedro", edad: 35 },
  { nombre: "Sofía", edad: 12 }
];

// Usá .filter() para quedarte con los mayores de 18
```

Pista: `filter` espera una función que devuelva `true` o `false`. Si devuelve `true`, el elemento se incluye en el nuevo array.

✔ Solución

```javascript
const personas = [
  { nombre: "Ana",   edad: 22 },
  { nombre: "Luis",  edad: 15 },
  { nombre: "Pedro", edad: 35 },
  { nombre: "Sofía", edad: 12 }
];

const mayores = personas.filter(p => p.edad >= 18);

console.log(mayores);
```

Nivel Intermedio

Los siguientes ejercicios combinan varios temas: objetos, destructuring, arrays, métodos y lógica condicional. Son los más parecidos a problemas reales.

Ejercicio 10 • Total del carrito

Intermedio | forEach | Destructuring | Template strings

Enunciado

Dado un array de productos (con `nombre`, `precio` y `cantidad`), calculá el **total a pagar** sumando `precio * cantidad` de cada uno. Mostrá cada línea del detalle y el total final.

Salida esperada

```
Remera x 2 = $10000
Pantalón x 1 = $15000
Zapatillas x 1 = $30000
──────────────────
TOTAL: $55000
```

Código inicial

```
const carrito = [
  { nombre: "Remera",     precio: 5000,  cantidad: 2 },
  { nombre: "Pantalón",   precio: 15000, cantidad: 1 },
  { nombre: "Zapatillas", precio: 30000, cantidad: 1 }
];

// Recorré el carrito, mostrá cada línea y el total final
```

Pista: Declará `let total = 0;` antes del bucle. Dentro de `forEach`, usá destructuring `({ nombre, precio, cantidad })` para escribir más limpio. En cada vuelta, sumá `precio * cantidad` a `total`.

✔ Solución

```javascript
const carrito = [
  { nombre: "Remera",     precio: 5000,  cantidad: 2 },
  { nombre: "Pantalón",   precio: 15000, cantidad: 1 },
  { nombre: "Zapatillas", precio: 30000, cantidad: 1 }
];

let total = 0;

carrito.forEach(({ nombre, precio, cantidad }) => {
  const subtotal = precio * cantidad;
  total += subtotal;
  console.log(`${nombre} x ${cantidad} = $${subtotal}`);
});

console.log("--------------------");
console.log(`TOTAL: $${total}`);
```

Ejercicio 11 • Aprobados y desaprobados

Intermedio | filter | map | && / ||

Enunciado

Dado un array de alumnos con su `nombre` y `nota`, hacé lo siguiente:

1. Filtrá los alumnos con nota mayor o igual a 6 (aprobados).
2. Mostrá en consola un array con los **nombres** de los aprobados.
3. Mostrá cuántos alumnos en total están desaprobados.

Salida esperada

```
Aprobados: ['Ana', 'Pedro', 'Sofía']
Desaprobados: 2
```

Código inicial

```javascript
const alumnos = [
  { nombre: "Ana",     nota: 8 },
  { nombre: "Luis",    nota: 4 },
  { nombre: "Pedro",   nota: 6 },
  { nombre: "Marta",   nota: 3 },
  { nombre: "Sofía",   nota: 9 }
];

// 1. Filtrá aprobados (nota >= 6)
// 2. Sacá solo los nombres con map
// 3. Calculá la cantidad de desaprobados
```

Pista: Podés encadenar métodos: `alumnos.filter(...).map(...)`. Para los desaprobados, usá `filter` con `nota < 6` y mirá la propiedad `.length` del resultado.

Solución

```javascript
const alumnos = [
  { nombre: "Ana",     nota: 8 },
  { nombre: "Luis",    nota: 4 },
  { nombre: "Pedro",   nota: 6 },
  { nombre: "Marta",   nota: 3 },
  { nombre: "Sofía",   nota: 9 }
];

const aprobados = alumnos
  .filter(a => a.nota >= 6)
  .map(a => a.nombre);

const desaprobados = alumnos.filter(a => a.nota < 6).
    length;

console.log("Aprobados:", aprobados);
console.log("Desaprobados:", desaprobados);
```

Ejercicio 12 • Sistema de login

Intermedio | Objetos | Destructuring | Condiciones anidadas

Enunciado

Simulá un sistema de login. Dado un objeto `usuario` y una `contrasenaIngresada`, mostrá un mensaje **diferente** según qué condición falle:

Condición	Mensaje
Todo correcto	`Bienvenido, [nombre].`
Intentos fallidos ≥ 3	`Cuenta bloqueada.`
Cuenta no activa	`Cuenta inactiva.`
Contraseña incorrecta	`Contraseña incorrecta.`

Código inicial

```
const usuario = {
  nombre: "Lucia",
  contrasena: "abc123",
  activa: true,
  intentosFallidos: 2
};

const contrasenaIngresada = "abc123";

// Verificá las condiciones en orden y mostrá el mensaje
```

¿Por qué importa el orden?
Si chequeamos la contraseña *antes* que el bloqueo, un usuario con cuenta bloqueada recibiría el mensaje "contraseña incorrecta", dando información innecesaria. Siempre evaluá primero la condición más crítica.

Pista:

1. Usá destructuring para sacar las propiedades del objeto.
2. Verificá primero `intentosFallidos >= 3`.
3. Luego `!activa` (el `!` niega un booleano).
4. Luego `contrasenaIngresada !== contrasena`.
5. Si ninguna se cumple, el acceso es exitoso.

✔ Solución

```javascript
const usuario = {
  nombre: "Lucia",
  contrasena: "abc123",
  activa: true,
  intentosFallidos: 2
};

const contrasenaIngresada = "abc123";

const { nombre, contrasena, activa, intentosFallidos } = usuario;

if (intentosFallidos >= 3) {
  console.log("Cuenta bloqueada.");
} else if (!activa) {
  console.log("Cuenta inactiva.");
} else if (contrasenaIngresada !== contrasena) {
  console.log("Contraseña incorrecta.");
} else {
  console.log(`Bienvenido, ${nombre}.`);
}
```

Juego final • ¿Qué imprime este código?

¿En qué consiste?
Se presentan fragmentos de código JavaScript ya escritos. Tu desafío es leerlos con atención y **predecir qué imprime** `console.log` **sin ejecutar el programa**. Anotá tu respuesta antes de mirar la solución.

Ronda 1 • typeof

```
console.log(typeof "42");
```

¿Qué imprime? (a) number (b) string (c) 42 (d) Error

Respuesta: (b) string
Aunque el contenido parezca un número, está entre comillas, por lo que es un `string`.

Ronda 2 • Template strings

```
const x = 5;
const y = 3;
console.log(`${x} + ${y} = ${x + y}`);
```

¿Qué imprime? (a) 5 + 3 = 53 (b) 5 + 3 = 8 (c) x + y = x+y (d) Error

Respuesta: (b) 5 + 3 = 8
Dentro de `${ }` se evalúa la expresión. `x + y` es 8 porque son números (no strings).

Ronda 3 • Destructuring

```
const persona = { nombre: "Ana", edad: 25 };
const { nombre } = persona;
console.log(nombre);
```

¿Qué imprime? (a) `undefined` (b) `Ana` (c) `{nombre: "Ana"}` (d) Error

✔ **Respuesta: (b) Ana**
El destructuring extrae la propiedad `nombre` del objeto y la asigna a una variable con el mismo nombre.

Ronda 4 • Operadores lógicos

```
const edad = 17;
const tieneDni = true;
console.log(edad >= 18 && tieneDni);
```

¿Qué imprime? (a) `true` (b) `false` (c) `17` (d) Error

✔ **Respuesta: (b) false**
El operador `&&` requiere que **ambas** condiciones sean `true`. Como `edad >= 18` es `false`, el resultado es `false`.

Ronda 5 • map

```
const nums = [1, 2, 3];
const resultado = nums.map(n => n + 10);
console.log(resultado);
```

¿Qué imprime? (a) `[1, 2, 3]` (b) `[10, 20, 30]` (c) `[11, 12, 13]` (d) `undefined`

✔ **Respuesta: (c) [11, 12, 13]**
`map` aplica la función a cada elemento. A cada número le suma 10 y devuelve un nuevo array.

Ronda 6 • for of

```
const letras = ["a", "b", "c"];
let resultado = "";
for (const letra of letras) {
  resultado += letra.toUpperCase();
}
console.log(resultado);
```

¿Qué imprime? (a) abc (b) ABC (c) ["A","B","C"] (d) Error

✔ **Respuesta: (b) ABC**
En cada vuelta, `letra` toma los valores `"a"`, `"b"` y `"c"`. Se concatenan en mayúsculas a la variable `resultado`.

Ronda 7 • Desafío final

```
const productos = [
  { nombre: "A", stock: 0 },
  { nombre: "B", stock: 5 },
  { nombre: "C", stock: 0 },
  { nombre: "D", stock: 8 }
];

const disponibles = productos
  .filter(p => p.stock > 0)
  .map(p => p.nombre);

console.log(disponibles);
```

¿Qué imprime? (a) ["A","B","C","D"] (b) ["B","D"] (c) [5, 8] (d) ["A","C"]

✔ **Respuesta: (b) ["B","D"]**
Primero `filter` se queda con los productos cuyo `stock > 0` (B y D). Luego `map` extrae solo el `nombre` de cada uno.

Referencia rápida

Declaración de variables

Palabra	Cuándo usarla	¿Reasignable?
`let`	Valor que puede cambiar	Sí
`const`	Valor que no cambia	No
`var`	Forma vieja, mejor evitarla	Sí

Operadores de comparación

Operador	Significado	Ejemplo
`===`	Igual (valor y tipo)	`edad === 18`
`!==`	Distinto (valor o tipo)	`color !== "rojo"`
`>`	Mayor que	`nota > 7`
`<`	Menor que	`x < 0`
`>=`	Mayor o igual que	`edad >= 18`
`<=`	Menor o igual que	`temp <= 10`

`===` vs `==`
Siempre usá `===` (estricto). Compara valor **y** tipo. Por ejemplo: `"5" == 5` es `true`, pero `"5" === 5` es `false`.

Operadores lógicos

Operador	Significado	Resultado `true` cuando...
`&&`	Y (AND)	**ambas** condiciones son `true`
`||`	O (OR)	**al menos una** condición es `true`
`!`	NO (NOT)	la condición original es `false`

Métodos de arrays más usados

Método	¿Para qué sirve?
`forEach`	Recorre el array ejecutando algo en cada vuelta. No devuelve nada.
`map`	Crea un **nuevo array** transformando cada elemento.
`filter`	Crea un **nuevo array** con los elementos que cumplen una condición.
`find`	Devuelve el **primer** elemento que cumple una condición.
`includes`	Devuelve `true` si el array contiene el valor buscado.

Estructura `if` / `else if` / `else`

```
if (condicion1) {
  // Se ejecuta si condicion1 es true
} else if (condicion2) {
  // Se ejecuta si condicion1 es false Y condicion2 es true
} else {
  // Se ejecuta si ninguna condición anterior fue true
}
```

🚀 **¡Buena suerte a todos!**